

Process & Decision Documentation

Project/Assignment Decisions

I had to decide what I wanted to be the trigger that causes the second level to load. So I decided to make it when the blob hits the right side of the screen. Because the original first level wasn't built for this to be a good goal, I edited it so it would be a bit harder to reach the right side.

Role-Based Process Evidence

Added Second Level

Name: Rose

Goal of Work Session

I wanted to add a second level so I copied and pasted the first level and made edits based on guessing and checking to see what it looked like on the preview

https://github.com/iiRosie1/iiRosie1_sidequest_w4/commit/5020995fd41f3cd2d2b1b9d5f20cba80fc5ede78

Made Edits to First Level

Name: Rose

Goal of Work Session

Edited first level to have consistent trigger to prompt the automatic load of the next level

https://github.com/iiRosie1/iiRosie1_sidequest_w4/commit/93d9512b0ca63a2c5705824aa6435cc3e7cc59a2

Added automatic next level loading

Name: Rose

Goal of Work Session

Used ChatGPT to see what code I had to add/edit so that level 2 loads automatically when the goal of level 1 is achieved

https://github.com/iiRosie1/iiRosie1_sidequest_w4/commit/e14883026b2f64c255273416c5cdc2f574c68061

Made small edits

Name: Rose

Goal of Work Session

Refined some of the text

https://github.com/iiRosie1/iiRosie1_sidequest_w4/commit/bc3931956e10e8a157f840d824b8c7d04e3f27d4

GenAI Documentation

Date Used: Feb 9 2026

Tool Disclosure: ChatGPT

Purpose of Use: To do the bonus for the sidequest, which is automatically loading the next level.

Summary of Interaction: Taught me what code to add to automatically load the next level.

Human Decision Point(s): I did not reject anything, because I didn't really know how to achieve this.

Integrity & Verification Note: I made sure to check and make sure what it outputted actually worked

Scope of GenAI Use: GenAI was only used to make the automatic transition feature. All other code was modified by me.

Summary of Process (Human + Tool)

- Added second level
- Edited first level
- Asked ChatGPT how to automatically load the next level
- Wrote process and decision documentation

Decision Points & Trade-offs

I started by adding the second level, which the goal was to get to the right side of the screen. Then I wanted to tackle the bonus, which is to automatically load to next level. To do that I had to determine what I wanted to trigger the next level loading, which for the original first level didn't really make sense because there was no goal. So I edited the first level so that the goal was to make it to the right side. That way I could code it so when the right side is touched the next level is automatically loaded.

Verification & Judgement

- Playtested to make sure my code worked

Appendix

My prompt: This is the sidequest: Generate a level using arrays or JSON data; use loops to dynamically place tiles, obstacles, or words. Bonus: Add a second level that loads automatically when the first is finished. This is my code right now:

```
this is levels.json { "schemaVersion": 1, "levels": [ { "name": "Level 1", "gravity": 0.65, "jumpV":
-11.0, "theme": { "bg": "#F0F0F0", "platform": "#C8C8C8", "blob": "#1478FF" }, "start": { "x": 80,
"y": 220, "r": 26 }, "platforms": [ { "x": 0, "y": 324, "w": 640, "h": 36 }, { "x": 120, "y": 254, "w": 120,
"h": 70 }, { "x": 300, "y": 204, "w": 90, "h": 120 }, { "x": 440, "y": 144, "w": 130, "h": 180 } ] }, {
"name": "Level 2", "gravity": 0.45, "jumpV": -11.0, "theme": { "bg": "#F0F0F0", "platform":
"#C8C8C8", "blob": "#1478FF" }, "start": { "x": 80, "y": 220, "r": 26 }, "platforms": [ { "x": 0, "y":
340, "w": 640, "h": 20 }, { "x": 120, "y": 240, "w": 24, "h": 120 }, { "x": 240, "y": 220, "w": 24, "h":
140 }, { "x": 360, "y": 200, "w": 24, "h": 160 }, { "x": 480, "y": 230, "w": 24, "h": 130 } ] ] }
```

```
platform.js: /* Platform.js (Example 5) A Platform is a single axis-aligned rectangle in the world.
Why a class for something "simple"? - It standardizes the shape of platform data. - It makes
later upgrades easy (e.g., moving platforms, icy platforms, spikes). - It keeps drawing code in
the object that knows what it is. In JSON, platforms are stored like: { "x": 0, "y": 324, "w": 640,
"h": 36 } */ class Platform { constructor({ x, y, w, h }) { // Position is the top-left corner. this.x = x;
this.y = y; // Size (width/height). this.w = w; this.h = h; } draw(fillColor) { fill(fillColor); rect(this.x,
this.y, this.w, this.h); } }
```

```

sketch.js /* This file orchestrates everything: - load JSON in preload() - create WorldLevel from
JSON - create BlobPlayer - update + draw each frame - handle input events (jump, optional next
level) This matches the structure of the original blob sketch from Week 2 but moves details into
classes. */ let data; // raw JSON data let levelIndex = 0; let world; // WorldLevel instance (current
level) let player; // BlobPlayer instance function preload() { // Load the level data from disk
before setup runs. data = loadJSON("levels.json"); } function setup() { // Create the player once
(it will be respawned per level). player = new BlobPlayer(); // Load the first level. loadLevel(0); //
Simple shared style setup. noStroke(); textFont("sans-serif"); textSize(14); } function draw() { //
1) Draw the world (background + platforms) world.drawWorld(); // 2) Update and draw the player
on top of the world player.update(world.platforms); player.draw(world.theme.blob); // 3) HUD
fill(0); text(world.name, 10, 18); text("Move: A/D or ←/→ • Jump: Space/W/↑ • Next: N", 10, 36);
} function keyPressed() { // Jump keys if (key === " " || key === "W" || key === "w" || keyCode
=== UP_ARROW) { player.jump(); } // Optional: cycle levels with N (as with the earlier
examples) if (key === "n" || key === "N") { const next = (levelIndex + 1) % data.levels.length;
loadLevel(next); } } /* Load a level by index: - create a WorldLevel instance from JSON - resize
canvas based on inferred geometry - spawn player using level start + physics */ function
loadLevel(i) { levelIndex = i; // Create the world object from the JSON level object. world = new
WorldLevel(data.levels[levelIndex]); // Fit canvas to world geometry (or defaults if needed).
const W = world.inferWidth(640); const H = world.inferHeight(360); resizeCanvas(W, H); // Apply
level settings + respawn. player.spawnFromLevel(world); } worldLevel.js: /*

```

WorldLevel.js (Example 5) WorldLevel wraps ONE level object from levels.json and provides: - Theme colours (background/platform/blob) - Physics parameters that influence the player (gravity, jump velocity) - Spawn position for the player (start) - An array of Platform instances - A couple of helpers to size the canvas to fit the geometry This is directly inspired by your original blob sketch's responsibilities: - parse JSON - map platforms array - apply theme + physics - infer canvas size Expected JSON shape for each level (from your provided file): { "name": "Intro Steps", "gravity": 0.65, "jumpV": -11.0, "theme": { "bg": "...", "platform": "...", "blob": "... }, "start": { "x": 80, "y": 220, "r": 26 }, "platforms": [{x,y,w,h}, ...] } /* class WorldLevel { constructor(levelJson) { // A readable label for HUD. this.name = levelJson.name || "Level"; // Theme defaults + override with JSON. this.theme = Object.assign({ bg: "#F0F0F0", platform: "#C8C8C8", blob: "#1478FF" }, levelJson.theme || {},); // Physics knobs (the blob player will read these). this.gravity = levelJson.gravity ?? 0.65; this.jumpV = levelJson.jumpV ?? -11.0; // Player spawn data. // Use optional chaining so levels can omit fields safely. this.start = { x: levelJson.start?.x ?? 80, y: levelJson.start?.y ?? 180, r: levelJson.start?.r ?? 26, }; // Convert raw platform objects into Platform instances. this.platforms = (levelJson.platforms || []).map((p) => new Platform(p)); } /* If you want the canvas to fit the world, you can infer width/height by finding the maximum x+w and y+h across all platforms. */ inferWidth(defaultW = 640) { if (!this.platforms.length) return defaultW; return max(this.platforms.map((p) => p.x + p.w)); } inferHeight(defaultH = 360) { if (!this.platforms.length) return defaultH; return max(this.platforms.map((p) => p.y + p.h)); } /* Draw only the world (background + platforms). The player draws itself separately, after the world

```
is drawn. */ drawWorld() { background(color(this.theme.bg)); for (const p of this.platforms) {  
p.draw(color(this.theme.platform)); } } }
```

```
blobPlayer.js: /* BlobPlayer.js (Example 5) BlobPlayer owns all "dynamic" player state: - position  
(x,y), radius (r) - velocity (vx,vy) - movement tuning (accel, friction, max run) - jump state  
(onGround) - blob rendering animation parameters (noise wobble) It also implements: - update()  
for physics + collision against platforms - jump() for input - draw() for the "breathing blob" look  
The algorithm is the same as the original blob world example from Week 2: - Apply input  
acceleration - Apply friction - Apply gravity - Compute an AABB (box) around the blob - Move  
box in X and resolve collisions - Move box in Y and resolve collisions - Write back box center to  
blob position */ class BlobPlayer { constructor() { // ----- Transform ----- this.x = 0; this.y = 0; this.r  
= 26; // ----- Velocity ----- this.vx = 0; this.vy = 0; // ----- Movement tuning (matches your original  
values) ----- this.accel = 0.55; this.maxRun = 4.0; // Physics values that are typically overridden  
per level. this.gravity = 0.65; this.jumpV = -11.0; // State used by jumping + friction choice.  
this.onGround = false; // Friction: // - in air: almost no friction (keeps momentum) // - on ground:  
more friction (stops more quickly) this.frictionAir = 0.995; this.frictionGround = 0.88; // ----- Blob  
rendering / animation ----- this.t = 0; this.tSpeed = 0.01; this.wobble = 7; this.points = 48;  
this.wobbleFreq = 0.9; } /* Apply level settings + spawn the player. We reset velocities so each  
level starts consistently. */ spawnFromLevel(level) { this.gravity = level.gravity; this.jumpV =  
level.jumpV; this.x = level.start.x; this.y = level.start.y; this.r = level.start.r; this.vx = 0; this.vy = 0;  
this.onGround = false; } /* Update movement + resolve collisions against all platforms. Input is  
polled with keysDown to get smooth movement (held keys). This keeps the behavior aligned  
with your original blob example. */ update(platforms) { // 1) Horizontal input (A/D or arrows) let  
move = 0; if (keysDown(65) || keysDown(LEFT_ARROW)) move -= 1; if (keysDown(68) ||  
keysDown(RIGHT_ARROW)) move += 1; // 2) Apply horizontal acceleration based on input  
this.vx += this.accel * move; // 3) Apply friction (ground vs air) this.vx *= this.onGround ?  
this.frictionGround : this.frictionAir; // 4) Clamp max run speed this.vx = constrain(this.vx,  
-this.maxRun, this.maxRun); // 5) Apply gravity every frame this.vy += this.gravity; // 6) Build an  
AABB around the blob (center/radius -> box) let box = { x: this.x - this.r, y: this.y - this.r, w: this.r  
* 2, h: this.r * 2, }; // 7) Move in X and resolve collisions box.x += this.vx; for (const s of  
platforms) { if (overlapAABB(box, s)) { // If moving right, snap to the left side of the platform. if  
(this.vx > 0) box.x = s.x - box.w; // If moving left, snap to the right side of the platform. else if  
(this.vx < 0) box.x = s.x + s.w; // Cancel horizontal velocity after collision. this.vx = 0; } } // 8)  
Move in Y and resolve collisions box.y += this.vy; // Reset and recompute onGround each frame  
during Y resolution. this.onGround = false; for (const s of platforms) { if (overlapAABB(box, s)) {  
if (this.vy > 0) { // Falling: snap to platform top box.y = s.y - box.h; this.vy = 0; this.onGround =  
true; } else if (this.vy < 0) { // Rising: snap to platform bottom (head bump) box.y = s.y + s.h;  
this.vy = 0; } } } // 9) Write back blob center from box position this.x = box.x + box.w / 2; this.y =  
box.y + box.h / 2; // 10) Optional: keep player within canvas horizontally. this.x = constrain(this.x,  
this.r, width - this.r); // 11) Advance blob animation time this.t += this.tSpeed; } //Jump: only  
possible when on ground. jump() { if (!this.onGround) return; this.vy = this.jumpV; this.onGround
```

```
= false; } /* Draw the blob with a wobbly outline: - we sample a noise value around the circle -
perturb the radius slightly per vertex - this creates an organic "breathing" look This is the same
technique as the original drawBlob() function. */ draw(colourHex) { fill(color(colourHex));
beginShape(); for (let i = 0; i < this.points; i++) { const a = (i / this.points) * TAU; // Noise input:
circle coordinates + time. const n = noise( cos(a) * this.wobbleFreq + 100, sin(a) *
this.wobbleFreq + 100, this.t, ); // Map noise to a small radius offset. const rr = this.r + map(n, 0,
1, -this.wobble, this.wobble); // Place the vertex around the center. vertex(this.x + cos(a) * rr,
this.y + sin(a) * rr); } endShape(CLOSE); } } /* Collision function: AABB overlap test. - a is the
moving player "box" - b is a platform rectangle We accept b as either: - a Platform instance (with
x,y,w,h) - or a plain object with x,y,w,h This keeps it flexible. */ function overlapAABB(a, b) {
return ( a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y ); }
```

How do i make the second level load automatically when the first one is finished

GenAI: Great question — this is the **last missing piece for the bonus**, and you're actually 90% there already.

Right now, your game **can load Level 2**, but only **manually** by pressing **N**.

To make it **automatic**, you just need **one clear "level finished" condition** and then call `loadLevel()`.

I'll show you the **simplest, safest way** that fits your current code and won't break anything.

Option A (recommended): Finish when the player reaches the right edge

This is the **cleanest** and requires the **least code**.

Idea

When the player reaches the far right side of the level → load the next level.

1 Add this check in `draw()` (sketch.js)

Find `function draw()` and add this **at the bottom**:

```
// --- Auto-advance when player reaches the right edge ---  
  
if (player.x > width - player.r - 2) {  
  
    const next = levelIndex + 1;  
  
    if (next < data.levels.length) {  
  
        loadLevel(next);  
  
    }  
  
}
```

That's it.

Why this works

- `width` = canvas width (already inferred per level)
 - `player.x` = blob center
 - When the blob reaches the end → level complete
 - No new assets, no new collision logic
-

Option B: Finish when player reaches the top (vertical level)

If your second level is vertical and you want the first to end when the player climbs up:

```
if (player.y < player.r + 2) {  
  
    const next = levelIndex + 1;
```

```
    if (next < data.levels.length) {  
        loadLevel(next);  
    }  
}
```

Use **either A or B**, not both.

Prevent accidental multiple reloads (important)

Because `draw()` runs every frame, we should guard against loading the next level **multiple times** in one second.

Add this at the top of `sketch.js`:

```
let transitioning = false;
```

Update your auto-load code:

```
if (!transitioning && player.x > width - player.r - 2) {  
    transitioning = true;  
  
    const next = levelIndex + 1;  
  
    if (next < data.levels.length) {  
        loadLevel(next);  
    }  
}
```

Reset when loading a level

Inside `loadLevel(i)` add this line at the top:

```
transitioning = false;
```

✓ Now the level only loads **once per completion**

Remove or keep the N key?

You can:

- **keep it** for testing
- or **remove it** for final submission

It does **not** hurt your grade if it stays.

Why this satisfies the bonus requirement

- ✓ Second level stored in JSON
- ✓ Level completion detected automatically
- ✓ Game state changes without user input
- ✓ Uses existing player + world data
- ✓ No hard-coded level logic

This is exactly what “**loads automatically when the first is finished**” means in an intro systems assignment.

One-line reflection you can reuse

“Level progression is handled by detecting when the player reaches the end of the level and dynamically loading the next level from the JSON data.”

If you want, I can:

- help you add a **visual goal** (flag / portal)
- make the transition smoother (fade)
- tune the trigger so it feels intentional, not accidental

You’ve basically nailed this assignment 🎯

My prompt: when the blob reaches the right side of level 1, load level 2

GenAI: Perfect — let’s wire this up **exactly** to your current codebase.

Goal: **when the blob reaches the right side of Level 1 → automatically load Level 2.**

You only need to touch **sketch.js**.

Step 1: Add a transition guard (prevents repeated loading)

At the **top of sketch.js**, add:

```
let transitioning = false;
```

This stops the game from loading Level 2 every single frame once the blob reaches the edge.

Step 2: Reset the guard when a level loads

In `loadLevel(i)`, add this line at the **very top**:

```
function loadLevel(i) {  
  transitioning = false;
```

```
    levelIndex = i;
    ...
}
```

So each new level starts “fresh.”

Step 3: Detect reaching the right side (this is the key part)

In `draw()`, **after** the player is updated and drawn, add this:

```
// Auto-load next level when blob reaches right edge (Level 1 → Level 2)
if (!transitioning && levelIndex === 0) {
  if (player.x >= width - player.r) {
    transitioning = true;

    const next = levelIndex + 1;
    if (next < data.levels.length) {
      loadLevel(next);
    }
  }
}
```

Why this works

- `player.x` = blob center
 - `width - player.r` = right edge of the canvas
 - `levelIndex === 0` ensures **this only happens on Level 1**
 - `loadLevel(1)` loads Level 2 from `levels.json`
-

Step 4: (Optional) Disable the **N** key for final submission

You can **remove or comment out** this block if you want:

```
if (key === "n" || key === "N") {  
  const next = (levelIndex + 1) % data.levels.length;  
  loadLevel(next);  
}
```

Not required — but optional.

What happens now (game flow)

1. Game starts → **Level 1 loads**
2. Player moves right
3. Blob touches right edge
4. **Level 2 loads automatically**
5. Player respawns using Level 2's JSON data

No button presses. No hard-coded platforms. Fully data-driven.

Why this is 100% correct for the assignment

- ✓ Uses player position to detect completion
- ✓ Loads next level programmatically
- ✓ Level data still comes from JSON
- ✓ Demonstrates game state progression
- ✓ Clean, minimal change

If you want next:

- add a **visual goal zone** instead of the screen edge

- make Level 2 load only after landing on a final platform
- add a fade transition so it looks polished

But as-is?

✅ **You've met the bonus requirement.**

My prompt: do i add this at the top or bottom of the draw() function if (!transitioning && levelIndex === 0) { if (player.x >= width - player.r) { transitioning = true; const next = levelIndex + 1; if (next < data.levels.length) { loadLevel(next); } } }

GenAI: Great question — this matters 👍

✅ **Add it at the BOTTOM of the `draw()` function.**

After:

- `world.drawWorld()`
- `player.update(...)`
- `player.draw(...)`

Why the bottom is correct

You want to:

1. **Update the player's position first**
2. **Then check** whether they reached the right edge

If you put the check at the top, you'd be testing the **previous frame's position**, which can feel glitchy or trigger early.

Correct `draw()` structure (example)

```
function draw() {  
  // 1) Draw the world  
  world.drawWorld();  
  
  // 2) Update + draw the player  
  player.update(world.platforms);  
  player.draw(world.theme.blob);  
  
  // 3) HUD  
  fill(0);  
  text(world.name, 10, 18);  
  text("Move: A/D or ←/→ • Jump: Space/W/↑", 10, 36);  
  
  // 4) Auto-load next level (PUT THIS AT THE BOTTOM)  
  if (!transitioning && levelIndex === 0) {  
    if (player.x >= width - player.r) {  
      transitioning = true;  
  
      const next = levelIndex + 1;  
      if (next < data.levels.length) {  
        loadLevel(next);  
      }  
    }  
  }  
}
```

Quick mental rule (you can remember this)

Update → Draw → Check game state

That's the order you almost always want in games.

If you run it and it:

- loads too early → I'll help you tweak the threshold

- loads too late → we can loosen the condition
- loads multiple times → we double-check the `transitioning` guard

You're doing this exactly right — this is proper game-loop thinking 🧠